AÇIK KAYNAK PROJE GÜVENLİK VE MİMARİ DENETİM RAPORU

İncelenen Proje: Gitea (https://github.com/go-gitea/gitea)

Kullanılan Ortam: VirtualBox - Kali Linux

PROJE ÖZETİ VE HEDEF

Bu projenin amacı, karmaşık ve gerçek dünya standartlarında yazılmış açık kaynaklı bir projeyi "Güvenlik Uzmanı / Sistem Mimarı" perspektifiyle incelemektir. Bu doğrultuda, modern yazılım geliştirme süreçlerini (CI/CD, Konteyner mimarisi, karmaşık yetkilendirme) barındıran Go dili ile yazılmış **Gitea** projesini seçtim. İncelemelerimi, izole bir test ortamı sağlaması amacıyla VirtualBox üzerinde çalışan Kali Linux sanal makinemde gerçekleştirdim.

Aşağıda, istenen 5 temel aşamaya dair yaptığım incelemeler, uyguladığım pratik adımlar ve tespitlerim yer almaktadır.

ADIM 1: Kurulum Analizi ve Dış Kaynak Güvenliği (Reverse Engineering)

Uygulanan Adımlar ve Gözlemler:

Projeyi Kali Linux makineme git clone komutu ile çektikten sonra ilk olarak kurulum betiklerini inceledim. Derslerimizde üzerinde durduğumuz güvensiz curl <url> | bash mantığının bu projede kullanılıp kullanılmadığını kontrol ettim. Gitea'nın geleneksel bir install.sh yerine, modern bir yaklaşım olan Makefile ve Go modül mimarisini kullandığını tespit ettim.

Dış Kaynak Güvenliği ve Hash Kontrolü:

Yazılımın indirdiği kaynaklar oldukça güvenli bir mekanizma ile doğrulanmaktadır. Gitea, dışarıdan bağımlılık (paket) çekerken go mod download komutunu kullanıyor. Açık kaynak güvenliği açısından en kritik noktalardan biri olan imza (hash) kontrolünü, ana dizinde bulunan go.sum dosyası aracılığıyla otomatik olarak yapıyor. Dışarıdan çekilen her paketin SHA-256 kriptografik özeti bu dosyadakilerle karşılaştırılıyor. Eğer paket yolda değiştirilmişse (Man-in-the-Middle saldırısı vb.), hash değerleri uyuşmayacağı için sistem kurulumu durduruyor. Yani uygulama kesinlikle körü körüne dış paket çalıştırmıyor.

Dizin ve Yetki Analizi:

Standart kurulumda binary dosyası /usr/local/bin/gitea konumuna, konfigürasyonlar /etc/gitea, veriler ise /var/lib/gitea konumuna yerleşmektedir. Kurulum aşamasında sistem, güvenlik amacıyla bu dizinlere root yetkisi vermek yerine, sadece bu iş için kısıtlı yetkilerle oluşturulmuş git kullanıcısına (chown git:git) sahiplik devretmektedir.

ADIM 2: İzolasyon ve İz Bırakmadan Temizlik (Forensics & Cleanup)

Uygulanan Adımlar ve İspat Yöntemi:

Bu işlemi izole ve güvenli bir şekilde yapabilmek için sanallaştırma teknolojisinin avantajlarından faydalandım.

Önlem (Snapshot): Gitea'yı kurmadan hemen önce VirtualBox üzerinden Kali Linux makinemin bir "Snapshot"ını (Anlık Görüntü) aldım. Bu benim sıfır noktam oldu.

Kurulum ve Çalıştırma: Uygulamayı ayağa kaldırdım, veritabanını oluşturduğunu ve 3000 portunu dinlemeye başladığını netstat -tulpn komutuyla teyit ettim.

Temizlik İşlemi (Manuel): Sistemi iz bırakmadan temizlemek için sırasıyla;

- Arka plan servisini durdurdum: systemctl stop gitea
- Servis kaydını sildim: rm /etc/systemd/system/gitea.service
- Uygulama, config ve log dizinlerini sildim: rm -rf /usr/local/bin/gitea /etc/gitea /var/lib/gitea /var/log/gitea
- İzole kullanıcıyı kaldırdım: userdel -r git

İspat Aşaması: Manuel temizlik sonrası açık port kalmadığını tekrar netstat ile teyit ettim. Ancak arka planda gözden kaçan bir log veya bağımlılık kalıntısı kalmadığından tam olarak emin olmak için, işlem bitiminde makineyi en başta aldığım Snapshot'a geri döndürdüm. Bu sayede sistemin bit bazında, Gitea'nın hiç var olmadığı o temiz ana döndüğünü kesin olarak ispatlamış oldum.

ADIM 3: İş Akışları ve CI/CD Pipeline Analizi

Uygulanan Adımlar ve Gözlemler:

Reponun. github/workflows/ dizinini incelediğimde, projenin çok katmanlı bir CI/CD otomasyonuna sahip olduğunu gördüm. Örneğin, lint-backend.yml ve build.yml dosyalarını analiz ettim.

Biri projeye yeni bir kod gönderdiğinde (Push) veya kod birleştirme isteği açtığında (Pull Request), bu YAML dosyaları tetikleniyor. GitHub Actions sanal bir Ubuntu ortamı oluşturuyor, Go kurulumunu yapıyor ve kodun güvenli olup olmadığını, standartlara uyup uymadığını (golangci-lint aracı ile) otomatik olarak test ediyor. Testten geçemeyen kod, ana projeye dahil edilmiyor.

Webhook Nedir?

Webhook, Gitea gibi sistemlerde (veya genel CI/CD mimarilerinde) "olay güdümlü" (event-driven) çalışan bir bildirim mekanizmasıdır. Sürekli "yeni kod var mı?" diye sistemi meşgul etmek yerine,

bir olay gerçekleştiğinde (örneğin depoya yeni kod push edildiğinde), Gitea hedef sisteme (Örn: Jenkins veya GitHub Actions) bir HTTP POST isteği gönderir. Tam olarak işlevi şudur: "Benim depoma yeni bir kod geldi, test süreçlerini başlatabilirsin." Webhook, otomasyonun tetikleyici anahtarıdır.

ADIM 4: Docker Mimarisi ve Konteyner Güvenliği

Uygulanan Adımlar ve Gözlemler:

Proje dizinindeki Dockerfile ve özellikle Dockerfile.rootless dosyalarını inceleyerek uygulamanın konteyner mimarisini analiz ettim.

Docker İmajı ve Katmanları: Docker imajı, uygulamanın çalışması için gereken her şeyin (işletim sistemi kütüphaneleri, bağımlılıklar, ayar dosyaları ve kod) paketlenmiş, sadece okunabilir (read-only) halidir. Gitea imajı incelendiğinde; Alpine Linux taban katmanı üzerine inşa edildiği, ardından gerekli paketlerin eklendiği, çalışma dizinlerinin (WORKDIR) oluşturulduğu ve uygulamanın eklendiği (COPY) katmanlardan (layers) oluştuğu görülmektedir.

Konteyner Erişimi ve Güvenliklaştırma: Standart bir konteyner varsayılan olarak izole çalışsa da, içindeki işlemler root olarak başlarsa güvenlik riski oluşturur. Gitea repo'sundaki Dockerfile.rootless versiyonu bu yüzden vardır; konteynerin sistem içinde root haklarına erişmesini engeller. Ortamı en güvenli hale getirmek için:

- Rootless (yetkisiz kullanıcı) imajlar kullanılmalı,
- Dosya sistemleri sadece okunabilir (read-only) bağlanmalı,
- Hassas veriler ortam değişkenlerinde açık metin olarak değil, Docker Secrets ile yönetilmelidir.

VM ve Kubernetes Farkı:

VM (Sanal Makine): Donanımı sanallaştırır. Her sanal makinenin kendine ait büyük bir işletim sistemi çekirdeği (Kernel) vardır. Güvenli ama çok hantaldır.

Docker: İşletim sistemini sanallaştırır. Benim Kali Linux'umun (Host) çekirdeğini paylaşır. İzolasyon sağlar ama VM kadar ağır değildir, çok hızlı başlar.

Kubernetes (K8s): Docker konteynerlerini yöneten bir orkestra şefidir. Yüzlerce Gitea konteynerini aynı anda yönetebilir, trafiği paylaştırır ve çökenleri otomatik yeniden başlatır.

ADIM 5: Kaynak Kod ve Akış Analizi (Threat Modeling)

Uygulanan Adımlar ve Gözlemler:

Bir güvenlik teknolojileri öğrencisi olarak projeye dışarıdan bir tehdit aktörü (hacker) gözüyle yaklaştım. Uygulamanın giriş noktası (entrypoint) ana dizindeki main.go dosyasıdır. Kimlik doğrulama işlemleri ise routers/, services/auth/ ve models/auth/ dizinleri altında yönetilmektedir. Sistem; OAuth2, LDAP ve WebAuthn (2FA) gibi çeşitli auth yöntemlerini desteklemektedir.

Tehdit Modellemesi:

Hacker Veri Hedeflerini Nasıl Bulur? Kaynak kodu inceleyen bir saldırgan, veritabanı tablolarının oluşturulduğu models/ dizinine bakarak veritabanı şemasını çözer. routers/api/ dizinine bakarak uygulamanın dışarı açılan kapılarını (endpoint'leri) bulur. Kodda "hard-coded" (koda gömülmüş) gizli anahtarlar, zayıf kriptografi kullanılan şifreleme fonksiyonları veya yetki kontrolü (RBAC) yapılmadan doğrudan veriye erişim izni veren hatalı fonksiyonlar (IDOR zafiyetleri) arar.

Auth Mekanizmasına Saldırı Vektörleri: Bulduğum auth mekanizmalarına şu şekilde saldırılabilir:

- **Brute-Force (Kaba Kuvvet) & Credential Stuffing:** Eğer uygulamanın oturum açma sayfasında veya API endpoint'inde yeterli bir "Rate Limiting" (hız sınırlaması) veya Captcha yoksa, otomatize araçlar (örn: Hydra veya Burp Suite) ile binlerce şifre kombinasyonu saniyeler içinde denenerek hesaplar ele geçirilebilir.
- **Session Hijacking (Oturum Çalma):** Eğer Gitea oturum çerezlerini (Session Cookie) verirken HttpOnly veya Secure bayraklarını (flags) kod içinde aktif etmemişse, olası bir XSS (Cross-Site Scripting) zafiyetinde kullanıcıların oturumları çalınabilir.
- **OAuth Manipülasyonu:** Eğer GitHub veya Google ile giriş yapılan OAuth2 akışında redirect_uri parametresi sunucu tarafında sıkı bir kontrol (whitelist) ile doğrulanmıyorsa, kurbanın yetki token'ları (Token Theft / Open Redirect) saldırganın kendi kurduğu sahte sunucuya yönlendirilerek çalınabilir.

Sonuç:

Bu denetim raporu sayesinde, bir açık kaynak projenin sadece kodlardan ibaret olmadığını; ağ mimarisi, izolasyon süreçleri, otomatik test mekanizmaları ve güvenliğe yönelik defansif kodlama (secure coding) pratiklerinin bir bütün olduğunu uygulamalı olarak analiz etmiş bulunmaktayım.

Beyan ve Kapanış:

İşbu rapor, İstinye Üniversitesi Bilişim Güvenliği Teknolojisi programı müfredatında yer alan **Güvenli Web Yazılımı Geliştirme - BGT208** dersi vize/proje ödevi gereksinimleri kapsamında akademik amaçlarla hazırlanmıştır. Rapor içerisinde detaylandırılan tüm kurulum, tersine mühendislik, izolasyon, tehdit modelleme ve güvenlik denetimi işlemleri; tamamen yalıtılmış sanal test ortamlarında (VirtualBox - Kali Linux) ve etik siber güvenlik prensiplerine sıkı sıkıya bağlı kalınarak gerçekleştirilmiştir. Elde edilen bulgular, modern yazılım mimarilerinin güvenlik duruşunu anlamak ve defansif yetenekleri geliştirmek amacıyla belgelenmiş olup, gerçek dünya sistemlerine yönelik herhangi bir yetkisiz müdahale veya zararlı eylem içermemektedir.

Gerçekleştirilen bu çalışma, teorik güvenlik eğitiminin pratik sistem denetimi yetenekleriyle birleştirilmesinde temel bir adım niteliği taşımaktadır.

Öğrenci Adı Soyadı: Raşit ÇANKAYA

Öğrenci Numarası: 2420191006

Bölüm: Bilişim Güvenliği Teknolojisi (İÖ)

Ders: Güvenli Web Yazılımı Geliştirme - **BGT208**

Tarih: 05/04/2026